



## **More with loops, lists, and tuples**

Hello again! In this video, we'll consider more complex examples of different ways to work with loops, lists, and tuples. I'll also introduce a few new tools that are useful for data professionals. Let's return to the women's basketball team list of players from the previous video. In this example, we'll integrate string formatting, loops, tuples, and lists. Remember, we have a list of tuples, where each tuple contains the name, age, and position of a player. Let's define a function that extracts the name and position of each player into a list that we'll use to print the information.

We'll call the function "player position," and its argument will be a list of tuples that contain player information. Next, we'll instantiate an empty list that will hold our result, which we'll build as we loop through the data. Now we'll use a for loop to unpack the tuples in our list of players. The variables we assign in the for loop must align with the format of the tuples. There are three components of each tuple in our list: name, age, and position, so we need three variables in our for loop. If we tried to unpack the tuples using only two variables, like, "for name, age in players," the computer would throw an error. It wouldn't know what to do, because there are three elements of the tuple but we're only giving the computer two containers to put them in.

So we'll begin our for loop "for name, age, position in players." Then we'll use string formatting to append each name and position to the result list. Each string will include some positional formatting and a new line too. Finally, we'll call this function in a for loop. This for loop will iterate over the results list that is output by the function and print each one. Now we have a nicely formatted, easily readable table of players and positions. Here's another application of loops and lists.

This is an example of nested loops. A nested loop is when you have one loop inside of another loop. These loops create all the different domino tiles in a set of dominoes, which is a tile-based game played with numbered gaming pieces. Feel free to pause the video and try to figure out what's happening... We start with the numbers on the left side of the dominoes. These numbers represent the dots, or pips, on the domino. They range from zero to six.

For each number in this range, we'll run another loop to generate the pips on the right side of the domino. Then we insert the left number and right number into a formatted print statement. And here are the dominoes! Notice that in the first print statement we included a parameter called "end" whose value was a whitespace. When a print statement executes, by default it will end with a new line. So without this parameter, all the dominoes would have been printed in a vertical line, each one beneath the next. But when we set the end character to a whitespace, it prints a space between each domino instead.

Here's the same code, but instead of printing the dominoes as strings, it stores each one as a tuple of integers in a list called "dominoes." Now suppose we want to check the second number of the tuple at index four. We can do that by using indexing. Start

with the list we want to access, "dominoes," and put the index of the tuple we want to access in brackets. Then add another pair of brackets containing the index of the value within that tuple. What if we want to calculate the total number of pips on each domino? We can do that with a for loop that iterates over each tuple, sums the value at index zero and the value at index one, and appends the sum to a list.

But there's a much easier way of doing this. It's called a list comprehension. A list comprehension formulaically creates a new list based on the values in an existing list. Here's how it works. Begin by assigning a variable for our new list. We'll call this one "pips from list comp." For its value, create an empty list.

Now we basically write a for loop, only in reverse. We begin with the calculation that creates each element of the list. In this case, we want each element to be the total number of pips on the domino, which is the domino at index zero plus the domino at index one. Then we add a "for" statement. We can check to make sure it gave the same result as our for loop. They're the same! Note what happened.

This is why I said a list comprehension is like a for loop written in reverse. The "for" part of it is at the end of the statement and the computation is at the beginning. Both the for loop and the list comprehension do the same thing, but the list comprehension is much more elegant, and usually faster to execute too. Hopefully by now you can appreciate how powerful the building blocks of Python can be. I encourage you to explore this notebook on your own and play with the code to discover what happens when you add something here or change something there. Playing with code is one of the best ways to learn! I'll see you soon.

# List comprehension

Formulaic creation of a new list based on the values in an existing list

```
In [ ]: ❸ pips_from_loop = []  
         for domino in dominoes:  
             pips_from_loop.append(domino[0] + domino[1])
```

```
In [ ]: ❸ pips_from_list_comp = [domino[0] + domino[1] for domino in dominoes]
```